

Teoría de la Computación 2007-I

Nota de Clase 11

Favio E. Miranda Perea

29 de noviembre de 2006

1. ¿Qué es un algoritmo?

- ¿Podemos reconocer cuando un procedimiento sistemático es un algoritmo?
- ¿Podemos dar una definición precisa del concepto de algoritmo?
- ¿Por qué es importante tener una definición precisa (matemática) de algoritmo?
- Un algoritmo es una colección de instrucciones simples para realizar una tarea o problema particular (procedimientos o recetas)
- Si se tiene un algoritmo para un problema dado P significa tener una manera para calcular efectivamente o resolver P .
- Un algoritmo es un proceso *potencialmente* realizable
 - Las operaciones del proceso se pueden realizar inequívocamente.
 - El número de operaciones o pasos del proceso es finito.

1.1. Existencia y Formalización de Algoritmos

- Décimo problema de Hilbert: Hallar un proceso de acuerdo al cual pueda determinarse en un número finito de pasos (un algoritmo) si un polinomio dado tiene una raíz entera.
- Se creía que todo problema P tenía una solución algorítmica.
- Más aún se pensaba en la existencia de un algoritmo universal U que pudiera resolver todos los problemas matemáticos.
- Los intentos por hallar el algoritmo universal U fallaron. Tal vez U no existía.
- ¿Cómo probar la no existencia de U ?
- Era necesario definir el concepto de algoritmo de una manera precisa y hallar un formalismo para poder probar propiedades de los mismos.
- Un formalismo que intente representar el concepto de algoritmo debería ser preciso y libre de ambigüedades, simple y general.

1.2. Formalismos para representar el concepto de algoritmo

- Máquinas de Turing (Alan Turing, Cambridge 1936)
- Cálculo Lambda (Alonzo Church, Princeton 1936)
- Sistemas de Post (Emile Post)
- Funciones μ -recursivas (Gödel, Herbrand, Kleene)
- Lógica Combinatoria (Curry, Schönfinkel)
- Máquinas de registro (Sheperdson, Sturgis)

Sorprendentemente todos los formalismos han resultado equivalentes. Es decir, un problema tiene solución en un formalismo si y sólo si tiene solución en cualquiera de los otros. Tal afirmación es un teorema, o una serie de teoremas rigurosamente demostrados. Esta supuesta coincidencia nos lleva a conjeturar que existe una única noción de computabilidad, esto se expresa mediante la llamada Tesis de Church-Turing:

1.3. La Tesis de Church-Turing

La tesis de Church-Turing afirma que la noción intuitiva de algoritmo es capturada de manera exacta por la noción matemática de máquina de Turing.

Un problema es soluble algorítmicamente si y sólo si es soluble mediante una máquina de Turing

Es decir, las máquinas de Turing implementan a cualquier algoritmo. Equivalentemente, una función es computable si y sólo si es computable mediante una máquina de Turing.

Las siguientes observaciones son relevantes:

- La afirmación es una tesis indemostrable pues la noción de algoritmo es intuitiva.
- Por otro lado la tesis es refutable y se destruiría mostrando un algoritmo que no pudiera ser implementado en una máquina de Turing.
- Existen fuertes evidencias a favor de la tesis, algunas de ellas son:
 - Intuitivamente cualquier algoritmo detallado para el cálculo manual puede programarse en una MT.
 - La equivalencia con otros formalismos mas modernos.
 - Existen demasiados ejemplos a favor y por supuesto ningún contraejemplo.
 - La comunidad tanto en matemáticas como en ciencias de la computación acepta ampliamente la tesis.

1.4. Computabilidad y Complejidad

Aceptamos la tesis de Church-Turing bajo la cual nos interesa discutir lo siguiente para un problema específico P .

- Computabilidad: ¿Es P efectivamente computable?, es decir, ¿Podemos resolver P mediante una máquina de Turing ?
- Complejidad: ¿Es P eficientemente computable?, es decir, ¿Puede P calcularse o implementarse de manera eficiente?

1.4.1. Observaciones acerca de la computabilidad

- Según la tesis de Church-Turing, las máquinas de Turing pueden computar todo lo que es computable. Sin embargo existe solamente un número infinito numerable de MT.
- Por otra parte el número de funciones digamos $f : \mathbb{N} \rightarrow \mathbb{N}$ **no** es numerable, es decir, hay más funciones que máquinas de Turing.
- Por lo tanto deben existir funciones no computables.
- Ejemplo prominente: el problema de la detención (Halting problem)

1.4.2. Observaciones acerca de la complejidad

- La complejidad es acerca de los problemas no de las soluciones.
- La pregunta de interés es ¿Qué tan complejo es resolver el problema? y **no** ¿Qué tan complejo es implementar la solución?
- La complejidad se divide en clases de problemas que comparten ciertas características computacionales.
- **P**: problemas que pueden resolverse por una MT en tiempo polinomial.
- **NP**: problemas que pueden resolverse por una MT no-determinística en tiempo polinomial.
- El problema más famoso en complejidad es el problema P vs. NP .

2. Máquinas de Turing

2.1. Generalidades

- Las máquinas de Turing son máquinas idealizadas capaces de realizar cálculos más complicados que los autómatas finitos o de pila.
- Una MT consiste de una cinta potencialmente infinita dividida en sectores (cuadros) y una cabeza de lectoescritura.
- Cada sector de la cinta contiene un símbolo de cierto alfabeto de entrada o bien el símbolo blanco.
- La cabeza lee el sector y puede escribir sobre él.
- La cabeza puede moverse a izquierda o derecha.

Definición 1 Una máquina de Turing es una septupla:

$$M = \langle Q, \Sigma, \Gamma, \delta, q_0, \perp, F \rangle$$

donde

- $Q \neq \emptyset$ es un conjunto finito de estados.
- Σ es el alfabeto de entrada.
- Γ es el alfabeto de la cinta, el cual incluye a Σ , es decir, $\Sigma \subseteq \Gamma$.

- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{\leftarrow, \rightarrow\}$ es la función de transición
- $q_0 \in Q$ es el estado inicial.
- $\perp \in \Gamma$ es el símbolo blanco tal que $\perp \notin \Sigma$.
- $F \subseteq Q$ es el conjunto de estados finales, el cual podría ser vacío.

Analizemos ahora el funcionamiento de la función de transición:

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{\leftarrow, \rightarrow\}$$

La transición $\delta(q, a) = (p, b, D)$ significa:

- El estado actual es q y el símbolo a leer es a .
- La transición es hacia el estado p
- b es el símbolo escrito en lugar de a .
- La cabeza se mueve una celda según la dirección dada por $D \in \{\leftarrow, \rightarrow\}$. Dicho movimiento se realiza después de leer a y escribir b .

Por ejemplo $\delta(q, a) = (p, b, \rightarrow)$ significa que el estado actual es q y el símbolo a leer es a . Al ejecutarse la transición la cabeza borra a , escribe b en su lugar, el estado actual cambia a p y la cabeza se mueve una celda a la derecha.

Veamos algunos ejemplos:

- $M = \langle \{q_0, q_1\}, \{a, b\}, \{a, b, \perp\}, \delta, q_0, \perp, \{q_1\} \rangle$ ¿Qué lenguaje se acepta?

$$\delta(q_0, a) = (q_0, b, \rightarrow)$$

$$\delta(q_0, b) = (q_0, b, \rightarrow)$$

$$\delta(q_0, \perp) = (q_1, \perp, \leftarrow)$$

- $M = \langle \{q_0, q_1\}, \{a, b\}, \{a, b, \perp\}, \delta, q_0, \perp, \emptyset \rangle$ ¿Qué lenguaje se acepta?

$$\delta(q_0, a) = (q_1, a, \rightarrow)$$

$$\delta(q_0, b) = (q_1, b, \rightarrow)$$

$$\delta(q_0, \perp) = (q_1, \perp, \rightarrow)$$

$$\delta(q_1, a) = (q_0, a, \leftarrow)$$

$$\delta(q_1, b) = (q_0, b, \leftarrow)$$

$$\delta(q_1, \perp) = (q_0, \perp, \leftarrow)$$

La función de transición también puede darse mediante una tabla como en los siguientes ejemplos de máquinas que reconocen los lenguajes dados.

- $L = \{a^n b^n \mid n \geq 1\}$

δ	a	b	X	Y	$\perp$
q_0	$(q_1, X, \rightarrow)$			$(q_3, Y, \rightarrow)$	
q_1	$(q_1, a, \rightarrow)$	$(q_2, Y, \leftarrow)$		$(q_1, Y, \rightarrow)$	
q_2	$(q_2, a, \leftarrow)$		$(q_0, X, \rightarrow)$	$(q_2, Y, \leftarrow)$	
q_3				$(q_3, Y, \rightarrow)$	$(q_4, \perp, \rightarrow)$
q_4					

- $L = \{w \in \{0, 1\}^* \mid w \text{ tiene un número par de ceros} \}$

δ	0	1	$\perp$
q_0	$(q_1, 0, \rightarrow)$	$(q_0, 1, \rightarrow)$	$(q_f, \perp, -)$
q_1	$(q_0, 0, \rightarrow)$	$(q_1, 1, \rightarrow)$	
q_f			

En la transición $\delta(q_0, \perp) = (q_f, \perp, -)$, $-$ significa que la cabeza no se mueve.

2.2. La Máquina Estandar de Turing

Existen diversas definiciones de MT, sin embargo hay una máquina estandar, que es a la que nos referiremos si no se indica lo contrario. Dicha máquina tiene las siguientes características:

- La cinta es infinita en ambas direcciones.
- Se permite un número arbitrario de movimientos en cualquier dirección.
- La máquina es determinista, δ define a lo más un movimiento para cada estado y símbolo dados.
- No hay transiciones desde estados finales, es decir, transiciones $\delta(q, a)$ si $q \in F$.
- No hay un archivo especial de entrada o salida, se asume que la máquina contiene algo al final y al principio del proceso.

2.3. Configuraciones y Lenguaje de Aceptación

Definición 2 Una configuración o descripción instantanea es una cadena de la forma

$$a_1 a_2 \dots a_{k-1} q a_k a_{k+1} \dots a_n$$

determinada por:

- El estado actual de la unidad de control (cabeza) q .
- El contenido de la cinta $a_1 a_2 \dots a_n \in \Gamma^*$
- La posición de la unidad de control (dada por la posición de q)

Obsérvese que en la cadena sobre la cinta la parte a la izquierda de q se ha consumido mientras que la parte de la derecha debe aún procesarse. En particular una configuración inicial es de la forma q_0w con $w \in \Sigma^*$.

Definición 3 Un cómputo o paso de computación es el cambio de una descripción instantánea a otra mediante una transición dada por δ . Formalmente

$$uqav \vdash ubpv$$

si y sólo si $\delta(q, a) = (p, b, \rightarrow)$ y

$$ucqav \vdash upcbv$$

si y sólo si $\delta(q, a) = (p, b, \leftarrow)$

La relación $\vdash^*$ se define de la manera usual.

Los casos particulares que involucran el inicio y final de una cadena sobre la cinta se definen de la misma manera, usando el símbolo blanco $\perp$.

$$uqa \vdash ubp\perp$$

si y sólo si $\delta(q, a) = (p, b, \rightarrow)$

$$qav \vdash p\perp bv$$

si y sólo si $\delta(q, a) = (p, b, \leftarrow)$

Pueden existir cómputos bloqueados o infinitos:

- Cómputos bloqueados: el cómputo se bloquea porque la siguiente transición no está definida. En tal caso escribimos

$$uqv \not\vdash^*$$

- Cómputos infinitos: el cómputo entra en un ciclo infinito, saltando de derecha a izquierda por ejemplo. En tal caso escribimos

$$uqv \vdash^* \infty$$

Con la noción formal de cómputo podemos definir al lenguaje aceptado por una máquina M de la manera usual:

$$L(M) = \{w \in \Sigma^* \mid q_0w \vdash^* w_1q_fw_2 \quad q_f \in F\}$$

$L(M)$ es el conjunto de todas aquellas cadenas de entrada con las cuales la máquina se detiene en un estado final.

Las siguientes observaciones son importantes:

- A diferencia con los autómatas se acepta una cadena en el momento en que el proceso llega a un estado final.
- No es necesario consumir toda la cadena.
- Si no hay estados finales se acepta una cadena en el momento en que la máquina se detiene, es decir, se aceptan todas las cadenas cuyos cómputos se bloquean.

3. Otros modelos de MT

- Existen diversas variaciones en la definición de MT.
- Todas ellas resultan equivalentes, es decir, el poder de computación de cualquier modelo resulta equivalente al de la máquina estandar.
- Las variaciones son útiles para simplificar la presentación o programación de diversos problemas.

3.1. MT con cabeza lectora estacionaria

- Se permite que al leer y escribir un símbolo la cabeza no realice movimiento alguno.
- El conjunto de direcciones se amplía a $\{\leftarrow, \rightarrow, -\}$.
- La transición

$$\delta(q, a) = (p, b, -)$$

significa que la cabeza lee a , escribe b y no se mueve

- Tales transiciones pueden simularse mediante un nuevo estado y movimientos consecutivos a la izquierda y a la derecha.

3.2. MT con múltiples pistas

- Idea: la cinta se divide en multiples pistas.
- La función de transición es:

$$\delta : Q \times \Gamma^n \rightarrow Q \times \Gamma^n \times \{\leftarrow, \rightarrow\}$$

$$\delta(q, \langle a_1, \dots, a_n \rangle) = (p, \langle b_1, \dots, b_n \rangle, D)$$

3.3. MT con múltiples cintas

- Idea: se agregan más cintas a la máquina
- La función de transición es:

$$\delta : Q \times \Gamma^n \rightarrow Q \times (\Gamma \times \{\leftarrow, \rightarrow\})^n$$

$$\delta(q, \langle a_1, \dots, a_n \rangle) = (p, \langle b_1, D_1 \rangle, \dots, \langle b_n, D_n \rangle)$$

3.4. MT No-determinista

- Se permite más de una transición en un estado y símbolos dados. La función de transición se modifica como sigue:

$$\delta : Q \times \Gamma^n \rightarrow \mathcal{P}(Q \times \Gamma \times \{\leftarrow, \rightarrow\})$$

$$\delta(q, \langle a_1, \dots, a_n \rangle) = \{\langle p_1, b_1, D_1 \rangle, \dots, \langle p_n, b_n, D_n \rangle\}$$

- Las máquinas no-deterministas juegan un papel central en la teoría de la complejidad.

4. Otros usos de las Máquinas de Turing

Además de utilizarse para reconocer lenguajes, las máquinas de Turing tienen poder suficiente para generar un lenguaje o calcular funciones.

4.1. MT Generadoras

- Las MT tienen suficiente poder como para generar lenguajes y calcular funciones.
- Una MT M genera al lenguaje $L \subseteq \Sigma^*$ si
 - M comienza a operar con la cinta en blanco en q_0 .
 - Cada vez que M regresa a q_0 hay una cadena de L escrita sobre la cinta.
 - Eventualmente se generan todas las cadenas de L .

4.2. MT Calculadoras de funciones

La máquina de Turing $M = \langle \{q_0, q_f\}, \Sigma, \Gamma, \delta, q_0, \perp, \emptyset \rangle$ calcula una función $f : \Sigma^* \rightarrow \Gamma^*$ si

$$q_0 w \vdash^* q_f v \quad \text{donde } f(w) = v$$

Observaciones:

- No hay estados finales, el estado q_f se usa para detener la máquina, es decir, no hay transiciones desde q_f .
- El proceso se termina en $q_f v$, es decir la cabeza debe estar leyendo el primer símbolo de la salida v .

En particular nos interesa calcular funciones $f : \mathbb{N}^k \rightarrow \mathbb{N}$ para lo cual usamos el alfabeto $\sigma = \{0, 1\}$ representando al natural n con la cadena 1^n . El cero toma el papel de separador en funciones de más de un argumento.

Definición 4 Una función $f : \mathbb{N}^k \rightarrow \mathbb{N}$ es Turing-computable si existe una MT M tal que para cualesquiera $x_1, \dots, x_k \in \mathbb{N}$ se cumple una y sólo una de las siguientes propiedades:

- Si $f(x_1, \dots, x_k)$ está definida entonces al recibir como entrada la cadena $1^{x_1}01^{x_2} \dots 01^{x_k}$, M se detiene siendo el contenido de la cinta la cadena $1^{f(x_1, \dots, x_k)}$
- Si $f(x_1, \dots, x_k)$ no está definida entonces M no se detiene al recibir como cadena de entrada $1^{x_1}01^{x_2} \dots 01^{x_k}$.

5. Lenguajes Recursivos y Recursivamente Enumerables

Para terminar el estudio de la jerarquía de Chomsky introducimos dos nuevas clases de lenguajes de suma importancia.

Definición 5 Un lenguaje L es recursivamente enumerable si es reconocido por una máquina de Turing M , es decir, si existe una máquina de Turing M tal que $L = L(M)$.

Definición 6 Un lenguaje L es recursivo si es reconocido por una máquina de Turing que siempre se detiene, es decir, si existe una máquina de Turing M tal que $L = L(M)$ y M se detiene con todas las cadenas de entrada.

El estudio de estas clases de lenguajes nos llevaría a un curso sobre computabilidad, aquí sólo mencionamos algunas de sus propiedades de cerradura así como su papel en la jeraquía de Chomsky.

Proposición 1 (Propiedades de Cerradura de Lenguajes recursivos y Rec. Enum.) Se cumplen las siguientes propiedades de cerradura:

- Si L es recursivo entonces $\bar{L}$ es recursivo.
- Si L, M son recursivos entonces $L \cup M$ es recursivo.
- Si L, M son rec. enumerables entonces $L \cup M$ es rec. enumerable.
- L es recursivo si y sólo si L y $\bar{L}$ son rec. enumerables.

5.1. Autómatas Linealmente Acotados y Gramáticas Sensibles al Contexto

Un autómata linealmente acotado (ALA) es una máquina de Turing que satisface las siguientes condiciones:

- El alfabeto de entrada Σ incluye dos símbolos especiales $[,]$ que sirven como marcas de fin de cinta izquierda y derecha respectivamente.
- La cabeza lectora no puede desplazarse más allá de dichos límites y no puede sobrecribir tales sectores.

Los autómatas linealmente acotados son importantes pues son el modelo de máquina correspondiente a los lenguajes sensibles al contexto.

Definición 7 Un autómata linealmente acotado M es una máquina de Turing adicionada con dos símbolos nuevos y especiales $[$ y $]$ ambos elementos de Σ

$$M = \langle Q, \Sigma, \Gamma, \delta, q_0, \perp, [,], F \rangle$$

con $[,] \in \Sigma$.

Las siguientes observaciones son relevantes:

- $[$ y $]$ son elementos de Σ que sirven como marcas de inicio y fin de cinta respectivamente.
- El lenguaje de aceptación es

$$L(M) = \{w \in \Sigma^* - \{[,]\} \mid q_0[w] \vdash^* w_1 q_f w_2 \quad q_f \in F\}$$

Es decir, Las marcas $[,]$ no son consideradas como parte de la cadena a procesar.

- Un ALA no puede moverse fuera de la cadena de entrada.

Proposición 2 (GSC $\Rightarrow$ ALA) Dada una gramática sensible al contexto G , existe un autómata linealmente acotado M tal que $L(G) = L(M)$. Es decir, los lenguajes sensibles al contexto son reconocidos por autómatas linealmente acotados.

Proposición 3 (ALA $\Rightarrow$ GSC) Si $L = L(M)$ es un lenguaje reconocido por un autómata linealmente acotado M entonces existe una gramática sensible al contexto G tal que $L(M) = L(G)$. Es decir, los lenguajes reconocidos por ALA son sensibles al contexto.

5.2. Máquinas de Turing y Gramáticas Irrestrictas

Para finalizar el estudio de la jerarquía de Chomsky mencionamos la equivalencia entre las máquinas de Turing y los lenguajes recursivamente enumerables.

Proposición 4 ($G0 \Rightarrow MT$) *Para toda gramática G de tipo 0 existe una máquina de Turing M tal que $L(M) = L(G)$. Es decir, los lenguajes tipo 0 son recursivamente enumerables.*

Proposición 5 ($MT \Rightarrow G0$) *Para toda máquina de Turing M existe una gramática G de tipo 0 tal que $L(G) = L(M)$. Es decir, los lenguajes recursivamente enumerables son lenguajes tipo 0.*

De esta manera completamos el estudio de la Jerarquía de Chomsky:

- Las gramáticas tipo 3 (regulares) son equivalentes a los autómatas finitos.
- Las gramáticas tipo 2 (libres de contexto) son equivalentes a los autómatas de pila (no-determinísticos).
- Las gramáticas tipo 1 (sensibles al contexto) son equivalentes a los autómatas linealmente acotados.
- Las gramáticas tipo 0 (irrestrictas) son equivalentes a las máquinas de Turing.

Obsérvese que los lenguajes recursivos no ocupan un lugar específico en la clase, sólo son parte de los lenguajes recursivamente enumerables.

6. La Máquina Universal

6.1. Máquinas de Turing vs. Computadoras

Una computadora es capaz de interpretar algoritmos arbitrarios y obtener la misma respuesta que cada algoritmo particular de manera que es una máquina útil para propósitos generales. Por otro lado las MT en son diseñadas para propositos particulares. De manera que el poder computacional de las MT no puede ser equiparable al de las computadoras actuales. En pocas palabras, las computadoras son programables, las MT no.

Sin embargo podemos preguntarnos si será factible pensar en la existencia de una MT que se comporte de la misma forma que una computadora de verdad, es decir, una MT que sea útil para propósitos multiples. Dicha máquina sería capaz de programar y ejecutar otras máquinas de Turing puesto que según la tesis de Church-Turing, un algoritmo es aquello que puede realizarse con una MT.

Tal máquina existe y se conoce como máquina universal de Turing (MUT):

- La MUT recibe como entrada una descripción de una MT, M y una cadena w y simula el comportamiento de M sobre w .
- Los datos de entrada M y w deben ser codificados de manera adecuada.

6.2. Codificación de MT

- Se fija un alfabeto de entrada Σ .
- Se asume $Q = \{q_0, \dots, q_n\}$ siendo q_0 el estado inicial y q_1 el **único** estado final.

- Si el alfabeto de cinta es de la forma $\Gamma = \{s_1, s_2, \dots, s_p\}$ con $s_1 = \perp$ entonces el símbolo s_i se codifica como 1^i .
- Las cadenas Γ^* se codifican separando cada símbolo con 0.
- Por ejemplo, si $\Gamma = \{\perp, a, b\}$ entonces codificamos $\perp := 1$, $a := 11$, $b := 111$.
- La palabra $bab\perp aa$ se codifica como: 01110110111010110110
- En general, si $w = s_{n_1}s_{n_2}\dots s_{n_k}$ la codificación de w es: $01^{n_1}01^{n_2}0\dots 01^{n_k}0$
- Si $Q = \{q_0, q_1, \dots, q_n\}$ el estado q_i se codifica, análogamente a los símbolos, mediante cadenas de unos pero con un símbolo más que el índice del estado

$$q_i \text{ se codifica como } 1^{i+1}$$

- Con la convención tomada, el estado inicial q_0 se codifica con 1 y el estado final q_1 con 11.
- El conjunto de direcciones $D = \{\rightarrow, \leftarrow, -\}$ se codifica como sigue:
 - $\rightarrow$ se codifica con 1.
 - $\leftarrow$ se codifica con 11.
 - $-$ se codifica con 111.
- Las transiciones $\delta(q_i, s_k) = (q_j, b_\ell, D)$ se codifican escribiendo en orden los códigos de q_i, s_k, q_j, b_ℓ, D delimitando la transición y separando cada componente por ceros:

$$01^{i+1}01^k01^{j+1}01^\ell01^n0$$

donde $n = 1, 2, 3$.

- Por ejemplo $\delta(q_2, s_3) = (q_0, s_5, \leftarrow)$ se codifica como

$$01^301^30101^501^20$$

- Una MT queda completamente determinada mediante su función de transición δ .
- La MT se codifica mediante la sucesión de los códigos de sus transiciones sin separaciones. Es decir, si $C_1, \dots, C_k$ son los códigos de todas las transiciones de M . Entonces M se codifica mediante

$$C_1C_2\dots C_k$$

- Obsérvese que no hay ambigüedad pues cada transición tiene exactamente seis ceros, además dos ceros consecutivos indican que inicia otra transición.

Por ejemplo la máquina M dada por:

$$\delta(q_0, a) = (q_2, b, \rightarrow) \quad \delta(q_2, b) = (q_3, c, \rightarrow)$$

$$\delta(q_3, a) = (q_1, c, \rightarrow) \quad \delta(q_1, b) = (q_3, \perp, -)$$

se representa usando la codificación $a := 11$, $b := 111$, $c := 1111$ como sigue:

010110111011101001110111011110111010

01111011011011110100110111011110101110

Las siguientes observaciones son relevantes:

- La codificación de una MT no es única, puesto que el orden de las transiciones no importa y un orden distinto genera una codificación distinta.
- De hecho si M tiene n transiciones, existen $n!$ codificaciones distintas para M .
- El proceso de codificación puede revertirse, no es difícil definir un algoritmo que decida si una secuencia binaria representa un código válido para MT y en tal caso lo decodifique.

6.3. Enumerabilidad de las MT

- En conclusión toda MT puede representarse como una cadena binaria.
- No todas las cadenas binarias representan MT válidas, por ejemplo, las cadenas que empiezan o terminan con 1 o las que tienen más de dos ceros consecutivos.
- Cada cadena binaria, representa por otra parte un número natural y viceversa. Es decir, hay tantas cadenas binarias como números naturales.
- De lo anterior se concluye que hay sólo un número numerable de MT.
- Las cadenas binarias pueden enumerarse en orden lexicográfico con $0 < 1$:

0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, 101, 110, 111, ...

- Cada MT figura varias veces en esta lista.
- Por motivos prácticos si una cadena no codifica directamente a una MT entonces acordamos que codifica a la máquina sin transiciones que acepta el lenguaje vacío.
- De esta manera cada cadena binaria codifica a una MT.
- Dado que las cadenas binarias son tantas como los números naturales y cada cadena codifica a una MT concluimos que sólo hay un número infinito numerable de MT.
- Por otro lado si consideramos las funciones $f : \mathbb{N} \rightarrow \mathbb{N}$ es bien sabido que son un número **no** numerable (tantas como números reales)
- De donde se concluye que existen funciones, que **no** pueden calcularse mediante una MT.
- Lo cual bajo la tesis de Church-Turing equivale a que existen funciones que no pueden ser calculadas mediante una computadora.

6.4. Descripción y funcionamiento de MUT

- La máquina universal de Turing $\mathcal{U}$ simula el comportamiento de cualquier MT sobre un alfabeto Σ dado.
- $\mathcal{U}$ recibe como entrada (el código de) una máquina M y una cadena w
- Los datos de entrada se representan como $M0w$, es decir, la entrada es una cadena binaria que consta del código de M y el código de w en orden separados por un cero.
- En tal caso la cadena tiene una única sucesión de tres ceros que separa a M y w .
- $\mathcal{U}$ es una máquina con tres cintas y alfabeto $\Gamma = \{0, 1, \perp\}$.
- La primera cinta recibe el código de una máquina cualquiera M .
- La segunda cinta recibe el código de una cadena w .
- La tercera cinta almacena el código del estado actual de M
- $\mathcal{U}$ examina el código de M para saber si es una máquina válida, en caso contrario $\mathcal{U}$ para sin aceptar.
- $\mathcal{U}$ lee las cintas dos y tres para buscar en la cinta uno la transición necesaria.
- Se aplica la transición modificando la cadena en la cinta dos y cambiando el estado en la cinta 3, como M lo haría. La unidad de control vuelve a la primera posición en las cintas uno y tres.
- Si al procesar w $\mathcal{U}$ se detiene en el único estado de aceptación entonces se acepta w , como el estado de aceptación es el mismo que el de M entonces M también aceptaba a w .
- Si no hay transición aplicable en la cinta uno o si $\mathcal{U}$ se detiene en un estado distinto al de aceptación w no se acepta, al igual que en M .
- $\mathcal{U}$ acepta a w si y sólo si M acepta a w .

6.5. Los Lenguajes Universal y Diagonal

Definición 8 (Lenguaje Universal) *El lenguaje aceptado por la máquina universal $\mathcal{U}$ se conoce como lenguaje universal, denotado $\mathcal{L}_{\mathcal{U}}$.*

$$\mathcal{L}_{\mathcal{U}} = \{M0w \mid M \text{ acepta } w \in \Sigma^*\}$$

Por definición $\mathcal{L}_{\mathcal{U}}$ es un lenguaje recursivamente enumerable.

Definición 9 (Lenguaje Diagonal) *Consideremos la enumeración de las máquinas de Turing $M_1, M_2, \dots$ así como la enumeración de todas las cadenas de Σ^* , digamos $w_1, w_2, \dots, w_n, \dots$. Podemos entonces dar como entrada la i -ésima palabra w_i a la i -ésima máquina M_i . El lenguaje diagonal se define como:*

$$\mathcal{L}_D = \{w_i \mid w_i \text{ no es aceptada por } M_i\}$$

Es decir $\mathcal{L}_D$ contiene a la i -ésima cadena si y sólo si ésta no es aceptada por la i -ésima máquina.

Proposición 6 *El lenguaje diagonal $\mathcal{L}_D$ no es recursivamente enumerable*

Demostración. Si $\mathcal{L}_D$ fuera R.E. sería aceptado por una MT, digamos la k -ésima máquina M_k . En tal caso $\mathcal{L}_D = L(M_k)$ y podemos preguntarnos si $w_k \in \mathcal{L}_D$.

- Si $w_k \in \mathcal{L}_D$ entonces w_k no es aceptada por $M_k \Rightarrow w_k \notin L(M_k) = \mathcal{L}_D$.
- Si $w_k \notin \mathcal{L}_D$ entonces $w_k \notin \mathcal{L}_D$, lo cual implica que w_k es aceptada por $M_k \Rightarrow w_k \in L(M_k) = \mathcal{L}_D$.

Por lo que se tendría

$$w_k \in \mathcal{L}_D \text{ si y sólo si } w_k \notin \mathcal{L}_D$$

Lo cual es absurdo.

—

De manera que existen lenguajes que no son recursivamente enumerables como $\mathcal{L}_D$, estos lenguajes se escapan de la jerarquía de Chomsky.

Con respecto a los lenguajes recursivos podemos mencionar que existen lenguajes recursivamente enumerables que no son recursivos. En particular el lenguaje universal $\mathcal{L}_U$ no es recursivo.